



**MAIMAN
ELECTRONICS**

Laser Diode Driver MODBUS Library

Maiman Electronics

e-mail: info@maimanelectronics.com

website: www.maimanelectronics.com

Version: 1.0.0 - 2025

Python Version: 3.9 or higher

LASER DIODE DRIVER MODBUS LIBRARY DOCUMENTATION	2
OVERVIEW	2
KEY FEATURES	2
USE CASES	3
LIBRARY COMPONENTS	3
REQUIREMENTS	4
MODBUSDEVICE CLASS	5
CONSTRUCTOR	5
PUBLIC METHODS	5
DECORATED PARAMETER ACCESS	5
DEVICE STATE CONTROL	6
START/STOP DEVICE	6
MODBUSDEVICEFACTORY CLASS	6
CLASS ATTRIBUTES	7
STATIC METHODS	7
MODBUSCOMMUNICATION CLASS	8
CONSTRUCTOR	8
PUBLIC METHODS	8
MODBUSDEVICEMODEL CLASS	10
CONSTRUCTOR	10
PUBLIC METHODS	10
CONSTRUCTOR	ERROR! BOOKMARK NOT DEFINED.
PUBLIC METHODS	ERROR! BOOKMARK NOT DEFINED.
PRIVATE METHODS	ERROR! BOOKMARK NOT DEFINED.
PROPERTIES	ERROR! BOOKMARK NOT DEFINED.
YAML FILES USED	11
YAML FILE STRUCTURE	14
MODBUS_CONFIG.YAML FILE	16
STRUCTURE	16
SECTIONS OF INTEREST	17
LOGGER CLASS	17
CONSTRUCTOR	17
PUBLIC METHODS	18
PRIVATE METHODS	18
LOGEVENT DATACLASS	19
EXCEPTION: DEVICEERROR	19
CONSTRUCTOR	19
EXAMPLE OF USAGE	19
EXAMPLE 1: INITIALIZATION, REGISTER INTERACTION, STATE CHANGES, AND OPERATION CONTROL	20
EXAMPLE 2: STRESS TESTING MODBUS DEVICES VIA RS-485 HUB	22
Purpose	22
Scenario: Hub-Based Communication	22
Workflow Summary	23
Benefits for Hub Testing	23
Test Recommendations	23

Laser Diode Driver MODBUS Library Documentation

Overview

The **Laser Diode Driver MODBUS Library** is a Python-based library designed to enable robust and modular communication with laser diode and TEC controllers over **RS485 using the MODBUS RTU protocol**. It supports real-time control, monitoring, and configuration of parameters such as **current, voltage, frequency, temperature, and operational states**.

Unlike its UART predecessor, this version leverages register-based communication via the `minimalmodbus` module and a YAML-backed configuration system for unit conversion, divider mapping, and signed interpretation—making it compatible with a wide range of Modbus-compliant devices.

With built-in logging, automated device ID retrieval, and helper decorators for clean register access, the library streamlines the integration of laser diode drivers into both development and industrial environments. Whether scanning networks of drivers or performing rapid stress tests, the Modbus Library offers a reliable and extensible solution for hardware communication.

Key Features

- **MODBUS RTU Communication**
Seamlessly interface with laser diode and TEC controllers over **RS485** using **MODBUS RTU** protocol, powered by the `minimalmodbus` library. A dedicated `ModbusCommunication` class wraps standard operations like register read/write and multi-register commands.
- **YAML-Based Device Configuration**
Automatically load device-specific configuration from YAML files using the `DeviceConfig` class. Divider values, units, and signed value interpretation are all abstracted away, allowing transparent communication with various models.
- **Parameter Management via Decorators**
Easily set and retrieve parameters like **current, frequency, duration, and voltage** using clean decorator-based definitions (`@modbus_get`, `@modbus_set`) inside the `ModbusDevice` class.
- **Device State Monitoring & Control**
Retrieve raw bitmask status from the device and perform safety checks or state transitions. Includes helper methods like `is_operation_started()` or `start_device()` to simplify workflow logic.
- **Integrated Logging System**
Use the `Logger` class to capture device communication logs, debug information, warnings, and errors. Logs can be output to files during stress tests or integration testing for full traceability.

- **Factory-Based Initialization**

The `ModbusDeviceFactory` ensures consistent instantiation with Singleton logic, parameterized by port, slave address, timeout, and logger. Useful for dynamic network discovery and modular device construction.

- **Multi-Device Support**

Supports multiple MODBUS slaves across the same bus. Designed for stress testing and scan routines, enabling dynamic polling and robust handling of intermittent devices.

- **Error Handling & Resilience**

Includes automatic reconnection, timeout control, and error capturing to prevent script freezes and facilitate graceful fallback during communication failures.

- **Modular and Extensible**

Easily extend support to new parameters, device types, or register layouts. The class structure promotes clean overrides, injection of YAML logic, or redefinition of device behavior with minimal changes.

Use Cases

- **Device Calibration**

Precisely configure current, voltage, and temperature values by writing to device registers using MODBUS. This functionality is especially useful in calibration routines for production or maintenance.

- **Device Control and State Management**

Start or stop the laser diode, monitor device status bits, and enforce interlocks using methods such as `start_device()`, `stop_device()`, and `is_operation_started()`. All state transitions are handled through interpreted register bitmasks.

- **Automated Testing and Integration**

Integrate the library into automated testing frameworks to simulate and verify laser diode driver behavior under various operating conditions. The modular structure supports scaling to multi-device environments.

- **Diagnostic Logging and Telemetry**

Leverage built-in logging support to capture communication logs, raw values, state transitions, and error messages. This enables easier diagnostics and detailed traceability during development or field operation.

- **Multi-Device Network Support (RS-485)**

Communicate with multiple laser diode drivers over a shared RS-485 MODBUS bus. This makes the library suitable for setups involving chained devices or centralized monitoring systems.

Library Components

ModbusDeviceFactory. Factory class responsible for creating and managing instances of `ModbusDevice`. Implements Singleton behavior to ensure consistent use of device instances across the application. Supports dynamic initialization via port, slave address, timeout, and logger.

ModbusDevice. High-level interface for communicating with laser diode via MODBUS. Exposes clean methods for getting/setting device parameters, controlling device state, retrieving status bitmasks, and executing start/stop operations.

ModbusDeviceModel. Encapsulates the mapping of logical parameter names to MODBUS register addresses. Used internally by decorators like `@modbus_get` and `@modbus_set` to simplify register-based access to device features.

ModbusCommunication. Low-level communication layer wrapping the `minimalmodbus` library. Handles connection setup, reading and writing to registers, interpreting signed values, and custom MODBUS operations (e.g., multi-register reads/writes).

DeviceConfig. Loads and manages parameter metadata from UART-style YAML files. Provides dividers, units, and signed-flag values used for interpreting register data. Promotes compatibility between MODBUS and existing configuration infrastructure.

Logger. Modular logging system used across the library and test scripts. Supports multiple log levels (INFO, DEBUG, ERROR) and allows writing logs to file for debugging or traceability during stress testing and deployment.

Requirements

The *requirements.txt* file lists the external dependencies required to run and develop the Laser Diode Driver MODBUS library.

Core Dependencies:

- **minimalmodbus>=1.0.0** – MODBUS RTU communication over RS-485
- **pyserial>=3.5**: For serial communication with the laser diode driver hardware.
- **PyYAML>=6.0.1**: For parsing YAML configuration files.

Development and Testing Dependencies:

- **pytest>=8.3.3**: For running automated tests.

To install the dependencies, run:

pip install -r requirements.txt

This ensures all necessary packages are installed for both runtime and development environments.

ModbusDevice Class

The ModbusDevice class provides a high-level interface for interacting with a MODBUS-compatible laser diode or TEC controller. It handles register-based communication, parameter scaling, signed value interpretation, and device state control.

Constructor

__init__(self, port=DEFAULT_PORT, slave_address=DEFAULT_SLAVE_ADDRESS)

Initializes the device by opening the MODBUS connection, retrieving the device ID, and loading the YAML configuration for that specific device.

Parameters:

- **port (str):** COM port or RS-485 interface name (default 'COM6').
- **slave_address (int):** MODBUS slave address (default 2).

Public Methods

get_device_id(self) -> int

Reads and returns the device ID from the configured MODBUS register.

get_divider(self, param_name: str) -> float

Returns the divider for a parameter from the YAML file.

get_units(self, param_name: str) -> str

Returns the display unit for a given parameter (e.g., 'mA', 'V').

to_signed(self, raw: int) -> int

Converts a 16-bit MODBUS unsigned value into a signed integer.

to_signed_16bit(self, val: int) -> int

Converts 16-bit values using two's complement logic (used for temperatures).

Decorated Parameter Access

Each parameter method is decorated with *@modbus_get* or *@modbus_set* and mapped to a register using ModbusDeviceModel.

@modbus_get(REGISTER_CURRENT_MEASURED)

def get_current_measured(self, raw): ...

Operation Method Signature

Get current *get_current(self, raw) -> float*

Set current *set_current(self, value: float) -> int*

Get temperature *get_pcb_temperature_measured(self, raw) -> float*
Get frequency *get_frequency(self, raw) -> float*
Set frequency *set_frequency(self, value: float) -> int*

Device State Control

get_raw_status(state_name: str) -> int

Reads raw status value (bitmask) from a device register.

is_bit_set(state_name: str, bit_mask: int) -> bool

Checks whether a specific bit is set in the device status.

is_operation_started()

Returns True if the operation state bit is active.

is_current_set_internal()

Checks if internal current set mode is enabled.

is_enable_internal()

Returns True if enable signal is active.

is_external_ntc_denied()

Checks external NTC interlock condition.

is_interlock_denied()

Checks general interlock condition.

Start/Stop Device

start_device()

Sends MODBUS command (0x0008) to start the device operation by enabling the 'operation state' bit.

stop_device()

Sends MODBUS command (0x0010) to stop the device operation.

ModbusDeviceFactory Class

The ModbusDeviceFactory class is a Singleton factory responsible for creating and managing a single instance of the ModbusDevice class. This factory pattern ensures that only one Modbus-based device instance is used throughout the application, centralizing initialization logic and simplifying device reuse.

Class Attributes

`_instance: Optional[ModbusDevice]`

A private class-level attribute that stores the singleton instance of the ModbusDevice. It is initially set to None and populated when the device is first created.

Static Methods

`get_device(port: str = "COM6", slave_address: int = 1, timeout: float = 0.01, logger: Optional[Logger] = None) -> ModbusDevice`

Retrieves the singleton instance of the ModbusDevice class. If the instance does not yet exist, the method performs the full setup:

- Connects via MODBUS
- Retrieves the device ID from the appropriate register
- Loads configuration from YAML
- Instantiates and configures a ModbusDevice object

Once created, the same instance is returned for all future calls.

Parameters:

- **port (str):** The serial port (e.g., "COM6").
 - **slave_address (int):** The MODBUS address of the device (default 1).
 - **timeout (float):** Timeout value for MODBUS communication.
 - **logger (Optional[Logger]):** Optional logger instance for recording events.
-

Returns:

- A singleton instance of ModbusDevice.
-

Behavior:

- If `_instance` is `None`, the factory:
 - Connects to the hardware
 - Retrieves the device ID
 - Loads device-specific configuration
 - Creates and returns a fully initialized `ModbusDevice`
- If `_instance` already exists, it is returned directly.

ModbusCommunication Class

The `ModbusCommunication` class manages low-level communication with devices using the MODBUS RTU protocol over RS-485. It wraps the `minimalmodbus` library and provides methods for reading/writing single or multiple registers, as well as buffer management and connection control.

Constructor

`__init__(self, port: str, slave_address: int = 100, baudrate: int = 115200, timeout: float = 1.0)`

Initializes the MODBUS connection parameters including port, address, and timeout configuration.

Parameters:

- `port` (str): The serial port name (e.g., 'COM6').
- `slave_address` (int): The MODBUS address of the target device.
- `baudrate` (int): Communication speed (default: 115200).
- `timeout` (float): Timeout in seconds for read/write operations.

Public Methods

`connect(self)`

Opens the serial port connection. Raises a `ConnectionError` if unable to connect.

`disconnect(self)`

Closes the serial port connection and sets the internal connection flag to `False`.

is_connected(self) -> bool

Returns True if the MODBUS port is currently open, False otherwise.

send_command(self, register: int, value: int)

Writes a single value to a given MODBUS register.

- **Parameters:**
 - register (int): The register address to write to.
 - value (int): The value to write.
- **Raises:**
 - IOError: On failed write operation.

receive_response(self, register: int, signed: bool = False) -> int

Reads a single register from the device.

- **Parameters:**
 - register (int): The address of the register.
 - signed (bool): Whether the returned value should be interpreted as signed.
- **Returns:**
 - Integer value read from the device.
- **Raises:**
 - IOError: On read failure.

send_multiple_registers(self, register: int, values: list[int])

Writes a list of integer values to consecutive registers.

- **Raises:**
 - IOError: On transmission error.

receive_multiple_registers(self, register: int, count: int, signed: bool = False) -> list[int]

Reads multiple registers starting from the specified address.

- **Returns:**
 - List of register values.
- **Raises:**
 - IOError: On read failure.

flush_input(self)

Flushes the input buffer of the MODBUS serial connection.

flush_output(self)

Flushes the output buffer of the MODBUS serial connection.

ModbusDeviceModel Class

The ModbusDeviceModel class provides access to the MODBUS register mappings for each parameter used by a device. It loads and stores register definitions from a YAML configuration file and is used by the ModbusDevice class to map high-level parameter names to low-level register addresses.

Constructor

```
__init__(self, yaml_path="config/modbus_config.yaml")
```

Initializes the device model by loading MODBUS register mappings from a specified YAML configuration file.

Attributes:

- register_map (Dict[str, int]): A dictionary mapping parameter names to MODBUS register addresses.

Public Methods

```
get_register(self, name: str) -> int
```

Retrieves the register address for a given parameter name.

- **Parameters:**
 - name (str): The parameter or state name.
 - **Returns:**
 - The register address as an integer.
-

```
@modbus_get Decorator
```

A decorator for creating parameter getter methods. It reads a value from the register mapped to parameter_name and passes the raw value to the decorated method.

Usage:

```
@modbus_get("voltage_measured")  
def get_voltage_measured(self, raw): ...
```

Behavior:

- Uses self.model.get_register(parameter_name) to resolve the register address.

- Calls `self.comm.receive_response(...)` to retrieve the raw value.
- Passes the result to the decorated method.

modbus_set Decorator

A decorator for creating parameter setter methods. It writes a value to the register mapped to `parameter_name`.

Usage:

```
@modbus_set("current")
def set_current(self, value): ...
```

Behavior:

- Uses `self.model.get_register(parameter_name)` to resolve the register address.
- Calls the decorated method to calculate the raw value (applying dividers, etc.).
- Sends the result to the device using `self.comm.send_command(...)`.

DeviceConfig Class

The DeviceConfig class manages the loading, validation, and retrieval of device configurations from a YAML file. It now also supports automatic normalization of parameter and state names for case-insensitive and format-insensitive access.

Constructor

```
__init__(self)
```

Initializes the DeviceConfig object with the default file path for the configuration file.

- **Attributes:**

- *file_path (str): The path to the YAML configuration file.*
- *config (Optional[dict]): The entire configuration loaded from the YAML file.*
- *device_config (Optional[dict]): The configuration specific to a particular device, loaded based on the device ID.*
- *_normalized_parameters (dict): Internal dictionary of normalized parameter names.*
- *_normalized_states (dict): Internal dictionary of normalized state names.*

Public Methods

```
load(self, device_id: int) -> dict
```

Loads the configuration for a specific device by ID, validates it, and builds normalized lookup maps.

- **Parameters:**

- *device_id (int): The ID of the device whose configuration is to be loaded.*
- **Returns:**
 - *A dictionary representing the device's configuration.*
- **Raises:**
 - *FileNotFoundError: If the YAML configuration file is not found.*
 - *ValueError: If the device ID is not found in the configuration or if the configuration is invalid.*

__repr__(self) -> str

Generates a string representation of the DeviceConfig object, showing the file path and a snippet of the configuration.

- **Returns:**
 - *A string representation of the DeviceConfig instance.*

Private Methods

_load_yaml(self) -> None

Loads the YAML configuration file into the config attribute.

- **Raises:**
 - *FileNotFoundError: If the YAML configuration file is not found.*
 - *ValueError: If there is an error in parsing the YAML file.*
 -

_validate_device_config(self, device_config: dict) -> None

Validates the structure of the device configuration, ensuring it contains required sections and valid codes.

- **Parameters:**
 - *device_config (dict): The configuration dictionary for a specific device.*
- **Raises:**
 - *ValueError: If required sections or parameter codes are missing.*

_get_section(self, section_name: str) -> dict

Helper method to retrieve a specific section of the device configuration.

- **Parameters:**
 - *section_name (str): The name of the section to retrieve (e.g., "error_codes", "parameters").*
- **Returns:**
 - *A dictionary representing the section of the configuration.*

- **Raises:**
 - `ValueError`: If the device configuration has not been loaded.

Properties

error_codes(self) -> dict

Returns a dictionary of error codes.

parameters(self) -> dict

Returns a dictionary of normalized parameter names. All names are lowercase and use underscores.

state_commands(self) -> dict

Returns a dictionary of normalized state command names.

tec_commands(self) -> dict

Returns a dictionary of TEC command configurations.

tec_state_commands(self) -> dict

Returns a dictionary of TEC state command configurations.

device_info(self) -> dict

Returns a dictionary of device info such as firmware version.

system_commands(self) -> dict

Returns a dictionary of system-level commands.

Note on normalization

All parameter and state names are automatically normalized:

- converted to lowercase
- spaces and dashes are replaced with underscores

This enables case-insensitive and style-insensitive access from user code.

Example: "Enable Internal" → "enable_internal"

YAML Files Used

This class is currently used to load the main UART-style device configuration file ``device_config.yaml``. It is also reused by the MODBUS library to retrieve dividers, units, and

command-related metadata even when registers are provided from a separate MODBUS-specific YAML file (``modbus_config.yaml``).

YAML File Structure

A typical YAML file is structured into sections based on the device's features and functionality.

Devices

The root of the YAML file contains a `devices` section, which lists devices by their unique ID. Each device entry includes:

- `name`: A human-readable name for the device.
- `id`: The device's unique identifier.
- `error_codes`, `parameters`, `state_commands`, `tec_commands`, and other sections for device settings.

Error Codes

The `error_codes` section lists various error conditions the device might encounter. Each error includes:

- `name`: A human-readable name for the error.
- `code`: The code returned by the device when the error occurs.
- `description`: A brief explanation of the error.

error_codes:

```
- name: "internal buffer overflow"
  code: "E0000"
  description: "Internal buffer overflow or command format invalid."
```

Parameters

The `parameters` section defines device parameters that can be controlled or measured. Each parameter includes:

- `name`: The name of the parameter.
- `code`: The code used to set or read the parameter.
- `min` and `max`: The minimum and maximum allowable values (often represented by codes).
- `divider`: A scaling factor used to convert the device's internal value to a human-readable form.
- `units`: The units of the parameter (e.g., "Hz", "mA").
- `measured`: The code for retrieving the measured value of the parameter.

parameters:

```
current:
  name: "current"
  code: "0300"
  min: "0301"
  max: "0302"
  divider: 10
  units: "mA"
  measured: "0307"
```

State Commands

The *state_commands* section manages device state transitions, such as enabling or starting operations. Each state command includes:

- name: The name of the state.
- code: The code used to read or modify the state.
- bits: A set of bit definitions within the state, each bit representing a sub-state of the device. Each bit includes:
 - name: A human-readable name for the bit.
 - on_command and off_command: Commands to turn the state on or off.
 - mask: A bitmask used to identify the state.
 - states: The possible values and their corresponding meaning.

state_commands:

```
state_of_device:
  name: "ofDevice"
  code: "0700"
  bits:
    "1":
      name: "operation state"
      on_command: "0008"
      off_command: "0010"
      mask: "0002"
      states:
        "0": "stopped"
        "1": "started"
```

TEC Commands

The *tec_commands* section defines the parameters and state management for the Thermo-Electric Cooler (TEC) component of the device. This includes temperature, TEC current, and voltage settings. Each TEC command includes:

- name: The name of the TEC command.
- measured: The code for retrieving the measured value.

- limit: The code for setting limits.
- divider: A scaling factor used to convert the internal value to a human-readable form.
- isSigned: A boolean indicating whether the value is a signed integer.

```
tec_commands:  
  tec_current:  
    name: "tecCurrent"  
    measured: "0A16"  
    limit: "0A17"  
    divider: 10  
    units: "A"  
    isSigned: true
```

System Commands

The system_commands section contains commands related to system-level operations, such as saving device parameters or resetting the device.

```
system:  
  save_parameters:  
    name: "saveParameters"  
    code: "0900"
```

modbus_config.yaml File

The modbus_config.yaml file contains register mappings used by MODBUS-compatible devices for parameter access and control. It defines the association between high-level parameter names and their corresponding MODBUS register addresses.

This YAML file is loaded by the ModbusDeviceModel class and is essential for enabling automated communication through the MODBUS protocol.

Structure

The file contains a single top-level key:

```
modbus_registers:  
  <parameter_name>: <register_address>
```

Each <parameter_name> represents a high-level logical device parameter (e.g., current, voltage_measured), and each <register_address> is a 16-bit hexadecimal MODBUS register.

Sections of Interest

- Basic Parameters: frequency, duration, current
 - Min/Max Constraints: frequency_min, current_max
 - Protection & Calibration: current_max_limit, ntc_b25_100_coefficient
 - Measurement Values: voltage_measured, tec_current_measured
 - State Registers: state_of_device, tec_state, lock_status
 - TEC Controls: temperature setpoints, limits, and PID tuning values
-

Logger Class

The `Logger` class provides an asynchronous logging mechanism for recording events and errors. It supports log rotation, UTF-8 encoding, and configurable logging levels. The class processes log asynchronously using a background thread and a queue.

Constructor

```
__init__ (self, log_file: str, max_file_size: int = 1024*100, backup_count: int = 5, enable_logging: bool = True)
```

Initializes the `Logger` class and sets up the logging configuration.

- **Parameters:**
 - `log_file` (str): The file path where logs will be stored.
 - `max_file_size` (int): The maximum size of the log file before rotating (default: 100 KB).
 - `backup_count` (int): The number of backup log files to maintain (default: 5).
 - `enable_logging` (bool): Whether logging is enabled (default: True).
 - **Attributes:**
 - `log_file` (str): The path to the log file.
 - `enable_logging` (bool): Flag to control whether logging is active.
 - `max_file_size` (int): The maximum size of a single log file in bytes.
 - `backup_count` (int): The number of backup log files to retain.
 - `log_queue` (Queue): A queue used for asynchronous log event processing.
 - `logger` (logging.Logger): The logger instance that manages file handling and formatting.
 - **Log Rotation:**
 - The logger uses a `RotatingFileHandler` to automatically rotate log files when they reach the specified `max_file_size`. Backup log files are maintained based on `backup_count`.
-

Public Methods

eventRecording(self, event: LogEvent)

Asynchronously records an event by placing it in the log queue for processing.

- **Parameters:**
 - event (LogEvent): The log event to be recorded.

errorRecording(self, message: str, error_code: Optional[int] = None)

Asynchronously records an error event with an optional error code.

- **Parameters:**
 - message (str): The error message to log.
 - error_code (Optional[int]): An optional error code associated with the error.

stop(self)

Stops the logging thread by sending a stop signal to the log queue.

start(self)

Restarts the logging thread if it was previously stopped.

path(self, new_path: str)

Updates the log file path and reinitializes the file handler with the new path.

- **Parameters:**
 - new_path (str): The new file path for the log file.
-

Private Methods

_process_log_queue(self)

Processes log events from the queue asynchronously. This method runs in a background thread and handles log events as they arrive.

_add_log_separator(self)

Adds a separator and timestamp to the log file at the start of each run. This makes it easier to distinguish between different application sessions.

```
_ensure_utf8(self, message: str) -> str
```

Ensures that a log message is properly encoded in UTF-8. If encoding fails, it falls back to ASCII.

- **Parameters:**
 - message (str): The log message to encode.
- **Returns:**
 - The encoded message as a UTF-8 string, or ASCII if encoding fails.

LogEvent Dataclass

The LogEvent dataclass represents a single logging event, containing the log level, message, and optional event data.

- **Fields:**
 - level (str): The logging level (e.g., "INFO", "ERROR", "DEBUG").
 - message (str): The message to log.
 - event_data (Optional[dict]): Optional additional data for the log event.

Exception: DeviceError

The DeviceError class is a custom exception that represents a device-related error. It stores the error code and description.

Constructor

```
__init__(self, code: str, description: str)
```

Initializes the DeviceError exception.

- **Parameters:**
 - code (str): The error code.
 - description (str): A description of the error.
- **Attributes:**
 - code (str): The error code associated with the exception.
 - description (str): A human-readable description of the error.

Example of Usage

This section provides practical examples of how to use the Laser Diode Driver MODBUS Library in Python. These scripts demonstrate how to initialize a device, configure communication, interact with device parameters, and perform stress tests.

Example 1: Initialization, Register Interaction, State Changes, and Operation Control

```
import os
import sys
import time
from datetime import datetime

# Setup environment path
parent_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), '..'))
sys.path.append(parent_dir)

from device import ModbusDevice
from logger import Logger

def test_all_required_modbus_steps():
    logger = Logger("modbus_documented_steps_test_log.txt")
    device = ModbusDevice(port="COM6", slave_address=1)

    try:
        device.comm.connect()
        print("☑ Connected to device.")

        # 1. Read basic parameters (with units)
        current = device.get_current()
        voltage = device.get_voltage_measured()
        tec_temp = device.get_tec_temperature_measured()
        pcb_temp = device.get_pcb_temperature_measured()
        print(f"Current: {current:.2f} {device.get_units('current')}")
        print(f"Voltage: {voltage:.2f} {device.get_units('voltage_measured')}")
        print(f"TEC Temp: {tec_temp:.2f} {device.get_units('tec_temperature_measured')}")
        print(f"PCB Temp: {pcb_temp:.2f} {device.get_units('pcb_temperature_measured')}")

        # 2. Write parameters
        device.set_current(500.0)
        device.set_frequency(100.0)
        device.set_duration(5.0)

        # 3. Read back to verify
        print(f"Set Current: {device.get_current():.2f} {device.get_units('current')}")
        print(f"Set Frequency: {device.get_frequency():.2f} {device.get_units('frequency')}")
        print(f"Set Duration: {device.get_duration():.2f} {device.get_units('duration')}")

        # 4. Device state before/after start
        state_before = device.get_raw_status("state_of_device")
        print(f"State before start: {hex(state_before)}")
        device.start_device()
```

```

    time.sleep(2)
    state_after = device.get_raw_status("state_of_device")
    print(f"State after start: {hex(state_after)}")

    # 5. Check key state bits
    print("Operation started:", device.is_operation_started())
    print("Enable active:", device.is_enable_internal())

    # 6. Read device identification
    print("Device ID:", device.get_device_id())
    print("Serial Number:", int(device.get_serial_number()))

    # 7. Extended register values
    print("Max Current Limit:", device.get_current_max_limit())
    print("Protection Threshold:", device.get_current_protection_threshold())
    print("Calibration Value:", device.get_current_set_calibration())

    # 8. TEC-related values
    print("TEC Voltage:", device.get_tec_voltage())
    print("TEC Current:", device.get_tec_current_measured())
    print("TEC Set Temperature:", device.get_tec_temperature_value())

    # 9. Safe stop
    device.stop_device()
    print("✅ Device stopped.")

except Exception as e:
    print(f"❌ Test failed: {e}")

finally:
    device.comm.disconnect()
    print("🔌 Disconnected.")

if __name__ == "__main__":
    test_all_required_modbus_steps()

```

Explanation:

- Initialization: Establishes communication with the MODBUS device and loads YAML-based configuration automatically using the retrieved device ID.
- Reading Parameters: Uses @modbus_get decorated methods and applies dividers and units for conversion to human-readable form.
- Writing Parameters: Uses @modbus_set methods for values like current, frequency, and duration.
- State Control: Enables and disables device operation via bitmask commands, checking the state_of_device register.
- Safety Check: Ensures safe operation via helper methods like is_operation_started() and is_enable_internal().
- Extended Values: Reads limit, protection, and calibration values for diagnostics or logging.

- **TEC Monitoring:** Retrieves temperature, voltage, and current from TEC-related registers.
 - **Logging:** Fully supports optional logging using the `Logger` class.
-

Example 2: Stress Testing Modbus Devices via RS-485 Hub

Purpose

This example performs a **robust polling stress test** for multiple Modbus-compatible laser diode drivers connected to a **shared RS-485 hub**. It demonstrates how to sequentially communicate with several devices sharing the same physical COM port (e.g., `COM6`) but using **different slave addresses**.

Scenario: Hub-Based Communication

In many industrial setups, laser diode drivers are **connected in parallel to a Modbus RTU hub**, using:

- A **shared communication line** (RS-485 bus)
- A **unique slave address** assigned to each device
- A **single master** (e.g., Python script via USB-to-RS485 adapter) that queries each device one at a time

This script stress-tests that type of setup by performing rapid communication with each device on the bus using minimal delay and timeout settings.

```
from factory.modbus_device_factory import ModbusDeviceFactory
from logger import Logger

def stress_test_modbus_devices(port="COM6", slave_addresses=[1, 2, 3], timeout=0.1,
    poll_interval=0.2, duration=30):
    logger = Logger("modbus_stress_test_log_1.txt")

    results = {addr: {"success": 0, "fail": 0, "total_delay": 0.0} for addr in slave_addresses}
    start_time = time.time()

    while time.time() - start_time < duration:
        for slave in slave_addresses:
            ModbusDeviceFactory._instance = None
            try:
                device = ModbusDeviceFactory.get_device(
                    port=port,
                    slave_address=slave,
                    timeout=timeout,
                    logger=logger
                )

            poll_start = time.time()
```

```
# Read critical parameters
current = device.get_current_measured()
device_id = device.get_device_id()
serial = device.get_serial_number()
status = device.is_operation_started()

poll_delay = time.time() - poll_start

results[slave]["success"] += 1
results[slave]["total_delay"] += poll_delay

print(f"✅ Slave {slave} | Current: {current:.2f} A | ID: {device_id} | Serial: {int(serial)}
| Running: {status}")
device.comm.disconnect()

except Exception as e:
    results[slave]["fail"] += 1
    print(f"❌ Slave {slave} failed: {e}")

time.sleep(poll_interval)
```

Workflow Summary

Step	Action
1	Iterate through all <code>slave_addresses</code> on the shared RS-485 hub
2	For each slave: connect, send Modbus request, and receive response
3	Read <code>current_measured</code> , <code>device_id</code> , <code>serial_number</code> , and operation status
4	Log response time and detect any failure
5	Repeat for all devices over the defined test <code>duration</code>

Benefits for Hub Testing

- **No interference** between devices, thanks to proper RS-485 addressing
 - Ensures that devices **respond within timeout**
 - Confirms **multi-device Modbus reliability**
 - Highlights any device that **consistently fails or slows down the bus**
-

Test Recommendations

- Use physical addresses like [1, 2, 3, 4, 5, 6, 7, 8] based on how devices are configured on the hub.

- Use `timeout` as low as `0.01s` to test edge-case responsiveness.
 - Log output using the `Logger` class to preserve test records.
-